Interfaces de Hardware - Proyecto 2do parcial ESP32-S3

Por Derek Monter y Renata Valle

Introducción

En este trabajo desarrollamos, en equipo de dos, una serie de prácticas con ESP-NOW, el protocolo de comunicación inalámbrica de Espressif que permite que dos placas ESP32 se manden datos directamente, sin necesidad de un router ni de una red WiFi. Cada quien trabajó con su propia placa ESP32-S3 (módulo "ESP32S3 Dev Module") y su propia computadora, usando Arduino IDE.

	Derek	Renata
Placa	ESP32-S3	ESP32-S3
Puerto	COM6	COM12
MAC (modo Station)	DC:DA:0C:26:C3:74	B8:1F:3F:DA:18:A8
Arduino IDE	2.3.10	2.3.8

Las prácticas fueron avanzando en complejidad: primero obtener la dirección MAC, después medir el alcance, luego enviar datos en una sola dirección, hacer broadcast, comunicarnos en ambos sentidos, cifrar los mensajes y, por último, subir todo a GitHub.

Práctica 00: Obtener la dirección MAC

Objetivo

En ESP-NOW cada placa se identifica por su dirección MAC. Para poder enviarle un mensaje a una placa específica hay que conocer su MAC, por eso esta práctica es la base de las que siguen (la necesitamos en las prácticas 2, 4 y 5).

Proceso

Cargamos el sketch `00_ObtenerMAC.ino` en cada una de las dos placas. El programa pone la placa en modo Station (`WIFI_STA`) y cada 3 segundos imprime su MAC por el monitor serie, tanto en formato legible como en el formato de arreglo que se pega directamente en el código.

```
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  delay(2000);
  WiFi.mode(WIFI_STA);
  delay(100);
}

void loop() {
  uint8_t mac[6];
  WiFi.macAddress(mac);
  Serial.print("MAC: ");
  Serial.println(WiFi.macAddress());
  Serial.printf("Para el codigo: {0x%02X, 0x%02X, 0x%02X, 0x%02X, 0x%02X,
0x%02X}\n\n",
                mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);
  delay(3000);
}
```

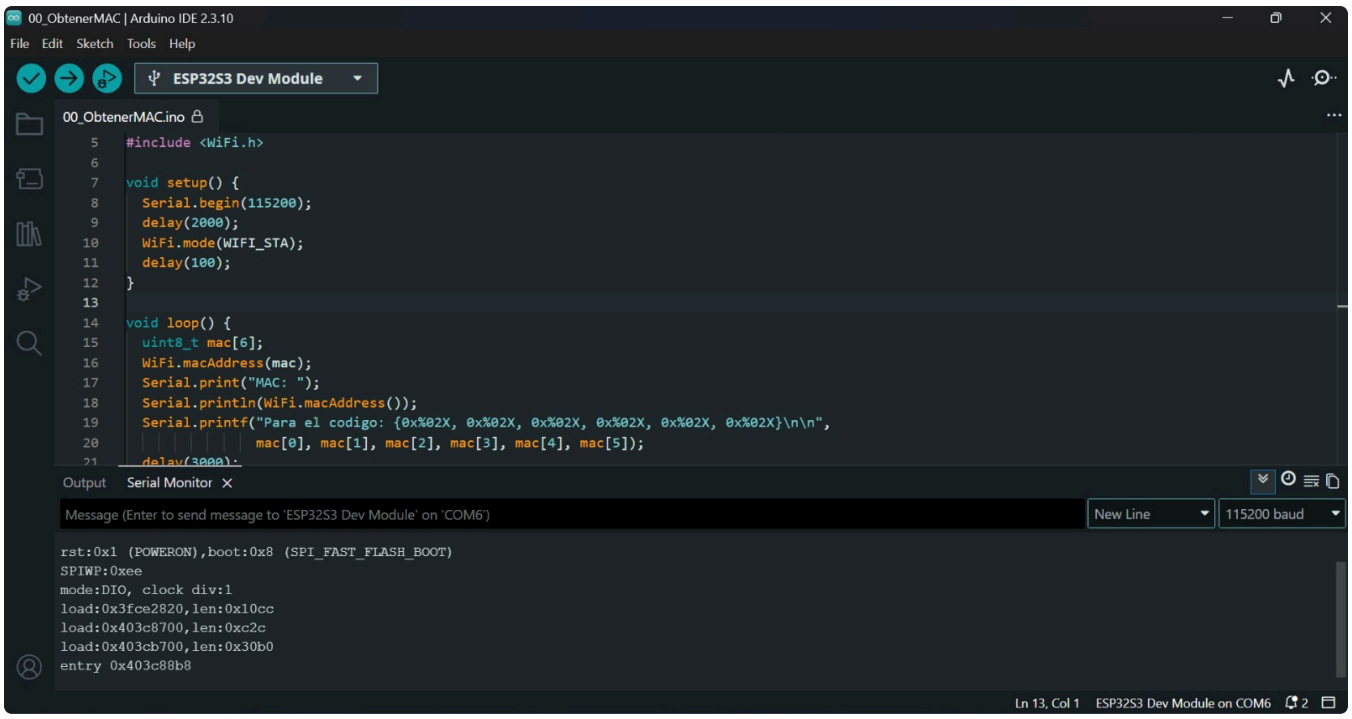
Resultado

El monitor serie (115200 baudios) mostró la MAC de cada placa:

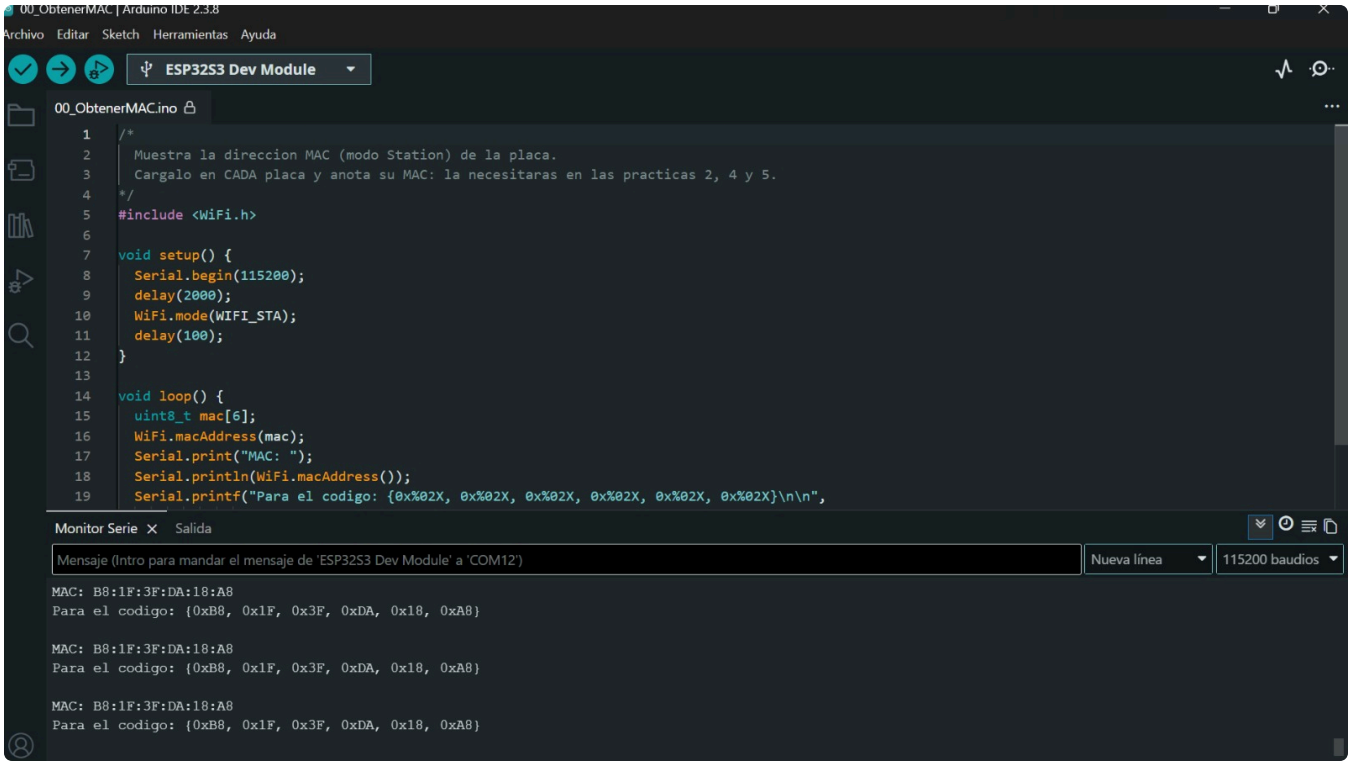
- Placa de Derek (COM6): `DC:DA:0C:26:C3:74` , es decir `{0xDC, 0xDA, 0x0C, 0x26, 0xC3, 0x74}`
- Placa de Renata (COM12): `B8:1F:3F:DA:18:A8` , es decir `{0xB8, 0x1F, 0x3F, 0xDA, 0x18, 0xA8}`

Anotamos ambas MAC porque cada placa necesita la de la otra en las prácticas siguientes.

Placa de Derek (COM6):



Placa de Renata (COM12):



Práctica 01: Distancia máxima

Objetivo

Medir hasta qué distancia sigue funcionando la comunicación ESP-NOW entre las dos placas, observando la intensidad de la señal (RSSI) y el porcentaje de paquetes perdidos.

Proceso

Una placa funciona como emisor y la otra como receptor. El emisor manda paquetes numerados (un contador `seq`) a la dirección de broadcast, de modo que no necesita conocer ninguna MAC. El paquete es una estructura mínima con solo el número de secuencia:

```
#include <WiFi.h>
#include <esp_now.h>

typedef struct __attribute__((packed)) {
  uint32_t seq;
} Paquete;

const uint8_t BROADCAST_MAC[6] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
uint32_t seq = 0;

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  if (esp_now_init() != ESP_OK) {
    Serial.println("Error al iniciar ESP-NOW");
    // ...
  }
  // ...
}
```

El emisor se quedó conectado a la computadora imprimiendo cuántos paquetes llevaba enviados (una línea cada 25 paquetes, con el estado del último envío).

El receptor es el que se aleja con la computadora. Como cada paquete lleva un número de secuencia, el receptor detecta los huecos entre un número y el siguiente para contar los paquetes perdidos, y con la información de recepción obtiene el RSSI. Imprime una línea por paquete:

```
typedef struct __attribute__((packed)) {
  uint32_t seq;
} Paquete;

uint32_t recibidos = 0, perdidos = 0, ultimaSeq = 0;
int rssi = 0;
volatile bool nuevo = false;
bool primero = true;

void OnDataRecv(const esp_now_recv_info_t *info, const uint8_t *data, int
len) {
  if (len != sizeof(Paquete)) return;
  // ...
}
```

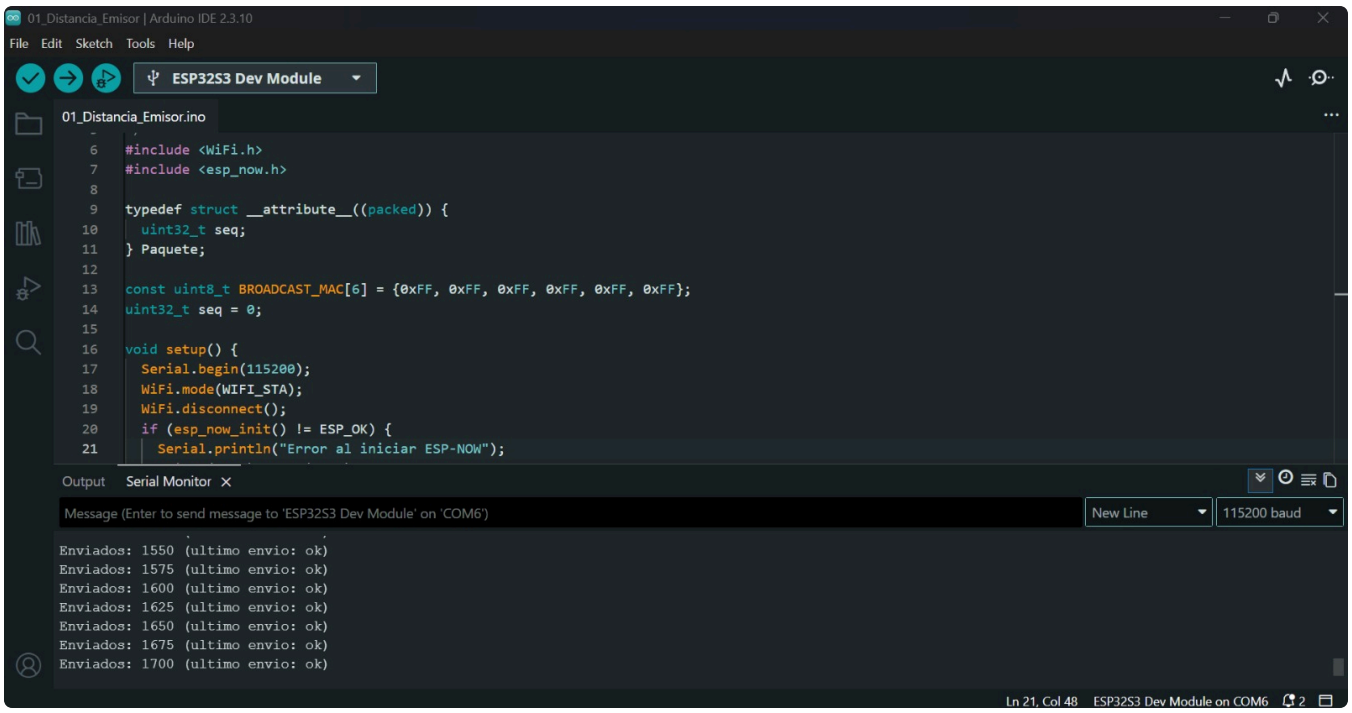
Resultado

Cerca, el emisor reportaba todos sus envíos como correctos (`Enviados: 1700 (ultimo envio: ok)`). Al alejarnos, el receptor mostró cómo se degradaba el enlace:

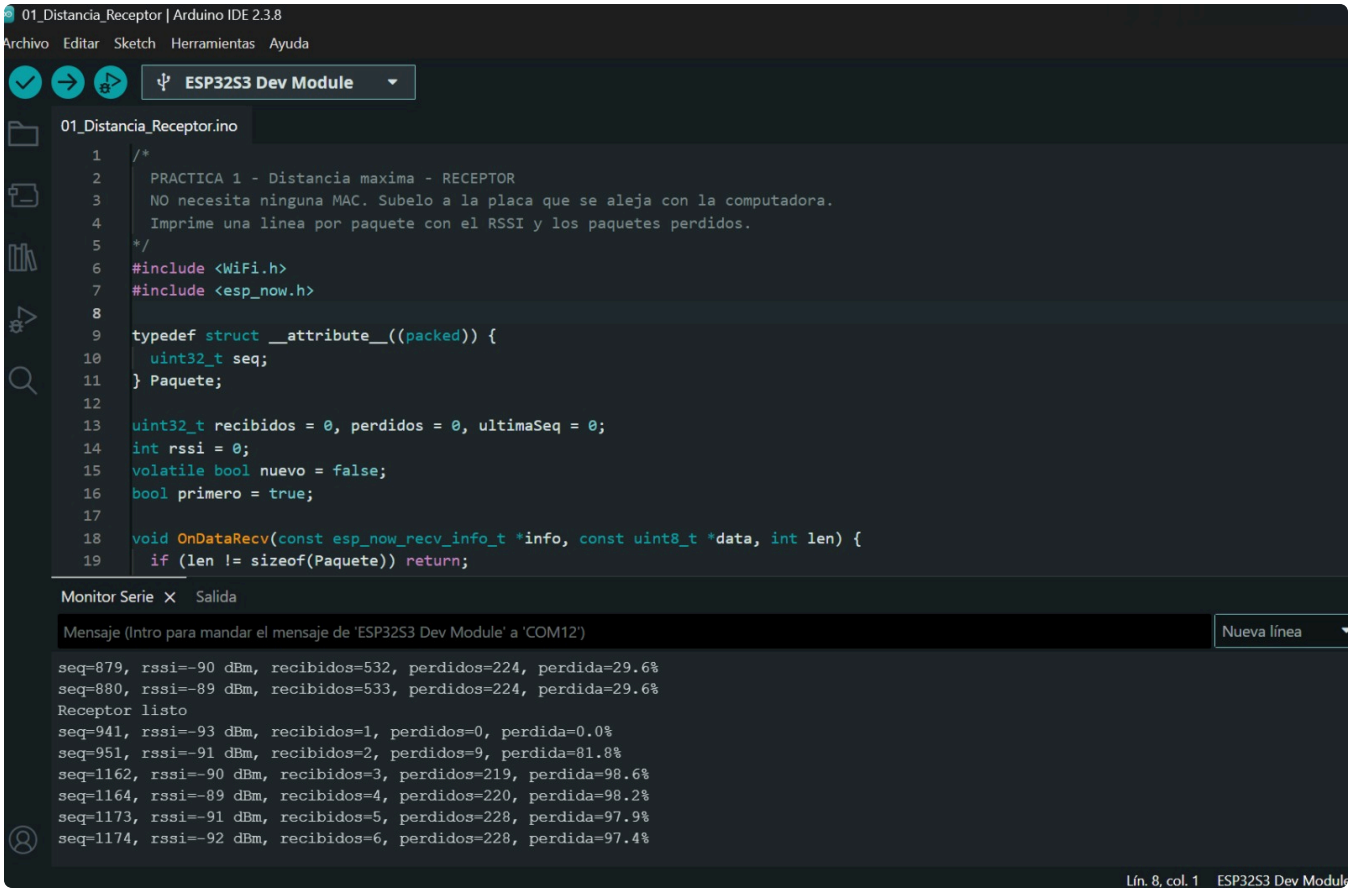
- A cierta distancia: RSSI cercano a -90 dBm y una pérdida de alrededor de 29.6 % (por ejemplo `seq=880, rssi=-89 dBm, recibidos=533, perdidos=224`).
- Más lejos: la pérdida subió a casi el 98 % con RSSI entre -89 y -93 dBm (por ejemplo `seq=1174, rssi=-92 dBm, recibidos=6, perdidos=228, perdida=97.4%`).

Esto nos dejó claro que, alrededor de los -90 dBm, el enlace ya es muy inestable y prácticamente deja de ser útil.

Emisor, placa de Derek (COM6):



Receptor, placa de Renata (COM12):



Práctica 02: Comunicación unidireccional

Objetivo

Enviar datos de una placa a otra en una sola dirección, usando la MAC del receptor, y confirmar la entrega.

Proceso

La placa de Derek (COM6) fue el emisor y la de Renata (COM12) el receptor. En el emisor lo único que se cambia es la línea `macReceptor`, donde se escribe la MAC de la placa receptora:

```
// >>> CAMBIA ESTO: MAC de la placa RECEPTORA <<<
uint8_t macReceptor[6] = {0xB8, 0x1F, 0x3F, 0xDA, 0x18, 0xA8};

typedef struct __attribute__((packed)) {
  uint32_t contador;
  float valor;
  char texto[32];
} Paquete;
```

El paquete ahora lleva tres datos: un contador, un valor decimal (que simula una lectura, por ejemplo una temperatura) y un texto. Como el envío es directo a una MAC concreta, ESP-NOW devuelve una confirmación (ACK), y el emisor lo informa en el monitor serie con `-> entregado (ACK recibido)`.

El receptor registra una función de recepción, valida que el tamaño del paquete sea el esperado, copia los datos y los imprime junto con el RSSI:

```
void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  if (esp_now_init() != ESP_OK) {
    Serial.println("Error al iniciar ESP-NOW");
    while (true) delay(1000);
  }
  esp_now_register_recv_cb(OnDataRecv);
  Serial.println("Receptor listo");
}

void loop() {}
```

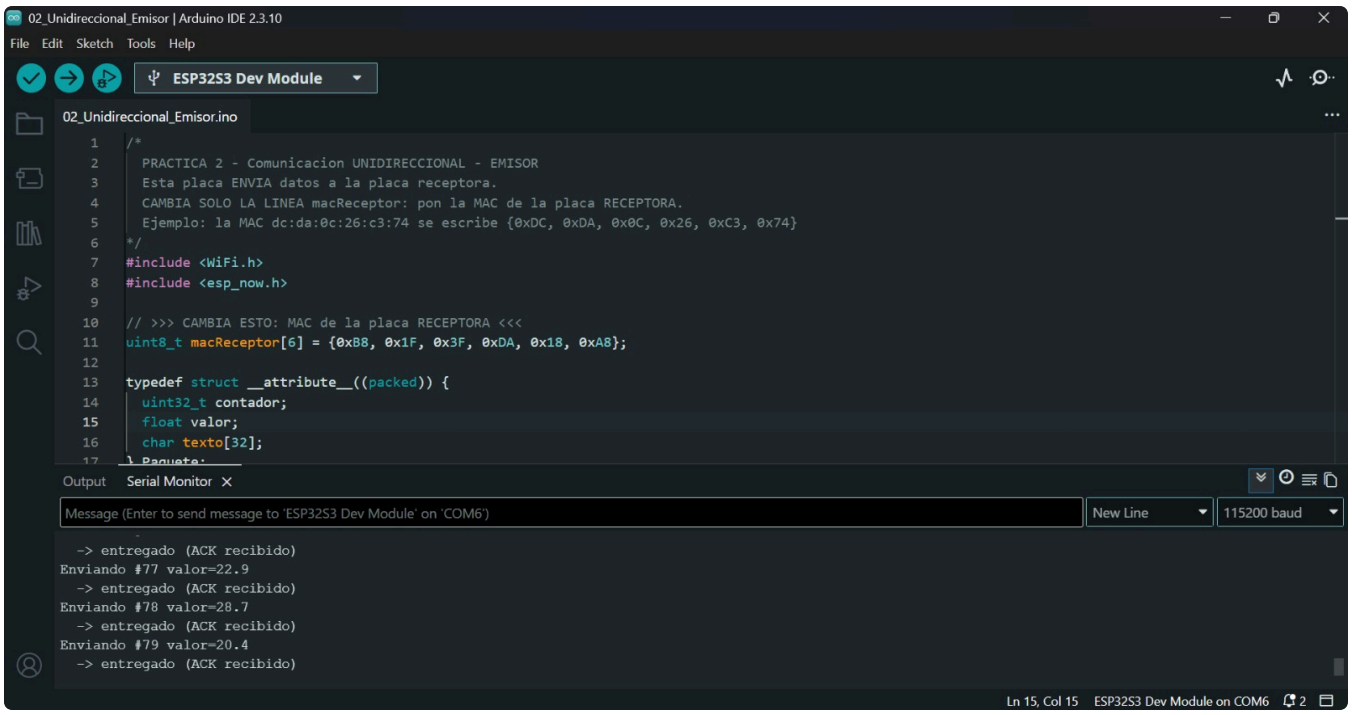
Resultado

El emisor mostró cada envío como entregado (`Enviando #79 valor=20.4` seguido de `-> entregado (ACK recibido)`), y el receptor recibió los mismos datos:

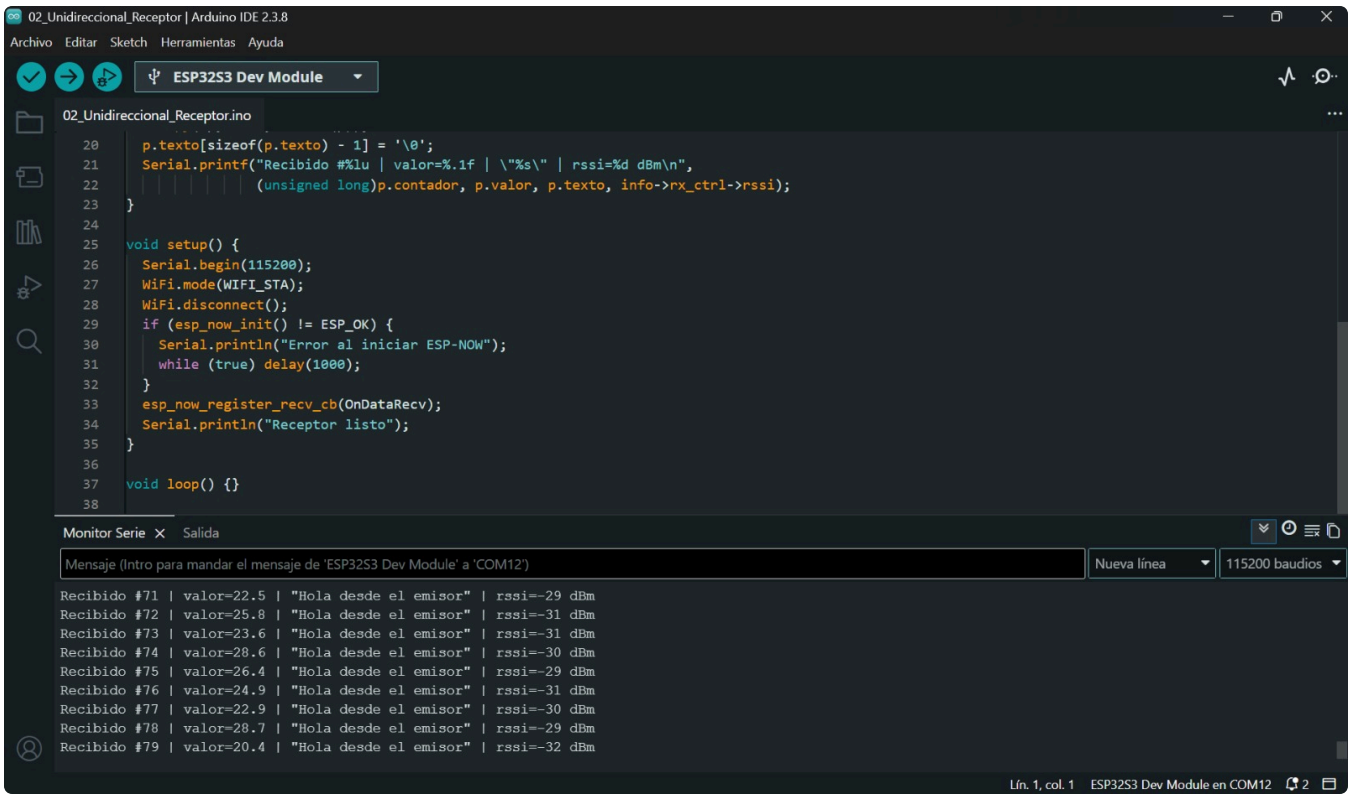
```
Recibido #79 | valor=20.4 | "Hola desde el emisor" | rssi=-32 dBm
```

Con las placas cerca, el RSSI se mantuvo entre -29 y -32 dBm, una señal muy fuerte. Los valores coinciden en ambas placas (por ejemplo, el #77 con valor 22.9 y el #78 con valor 28.7), lo que confirma que la comunicación unidireccional funciona.

Emisor, placa de Derek (COM6):



Receptor, placa de Renata (COM12):



Práctica 03: Broadcast

Objetivo

Enviar un mensaje a todas las placas cercanas a la vez, sin conocer la MAC de ninguna.

Proceso

En lugar de la MAC de un destinatario, el emisor usa la dirección de broadcast `FF:FF:FF:FF:FF:FF`. Cualquier placa que esté escuchando recibe el mensaje.

```
const uint8_t BROADCAST_MAC[6] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

typedef struct __attribute__((packed)) {
  uint32_t contador;
  char texto[32];
} Paquete;

void OnDataSent(const void *info, esp_now_send_status_t status) {
  Serial.println(status == ESP_NOW_SEND_SUCCESS ? "  -> enviado al aire (sin
ACK en broadcast)"
                                                : "  -> error al enviar");
}
```

Una diferencia importante con la práctica anterior: en broadcast no hay ACK, porque no hay un destinatario único que confirme. Por eso el mensaje del emisor dice "enviado al aire" y no "entregado".

El receptor no necesita ninguna MAC ni cambios. Además del mensaje, muestra la MAC de quien lo envió, tomada de `info->src_addr`:

```
void OnDataRecv(const esp_now_recv_info_t *info, const uint8_t *data, int
len) {
  if (len != sizeof(Paquete)) return;
  Paquete p;
  memcpy(&p, data, sizeof(p));
  p.texto[sizeof(p.texto) - 1] = '\0';
  const uint8_t *m = info->src_addr;
  Serial.printf("De %02X:%02X:%02X:%02X:%02X:%02X | #%lu | \"%s\" | rssi=%d
dBm\n", ...);
}
```

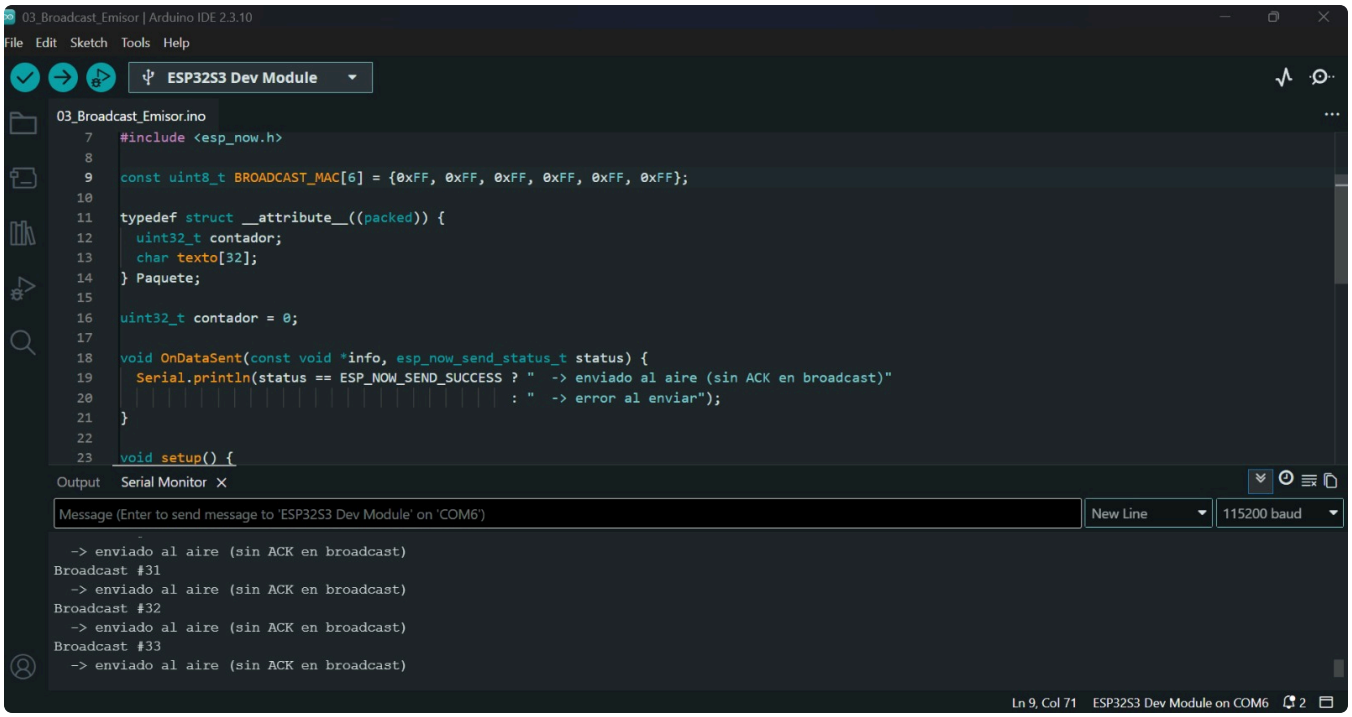
Resultado

El emisor (placa de Derek) fue contando sus broadcasts (`Broadcast #31` , `#32` , `#33` ...) y el receptor (placa de Renata) los recibió identificando de quién venían:

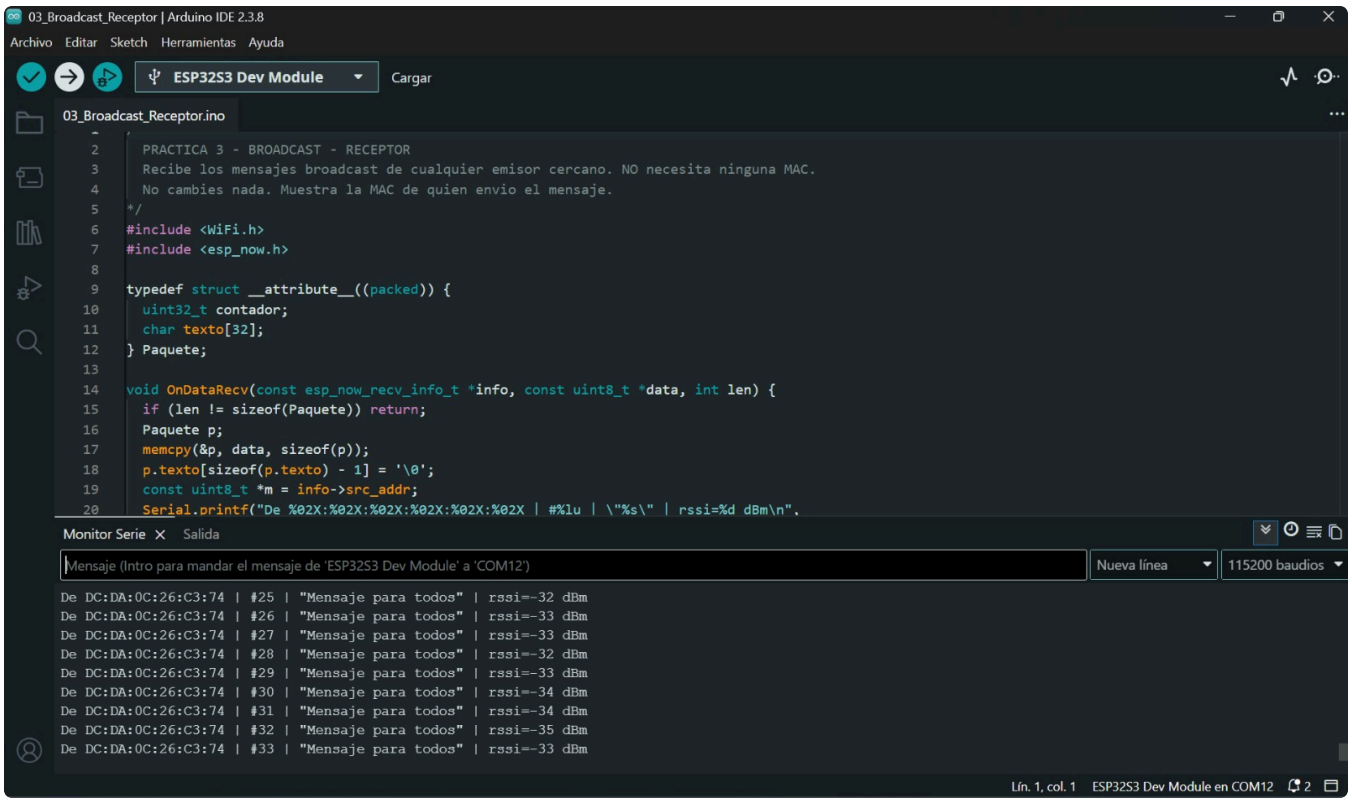
```
De DC:DA:0C:26:C3:74 | #33 | "Mensaje para todos" | rssi=-33 dBm
```

La MAC que aparece (`DC:DA:0C:26:C3:74`) es la de la placa de Derek, lo que confirma que el receptor identifica correctamente al emisor sin haberlo configurado de antemano.

Emisor, placa de Derek (COM6):



Receptor, placa de Renata (COM12):



Práctica 04: Comunicación bidireccional

Objetivo

Que las dos placas envíen y reciban al mismo tiempo, es decir, que cada una sea emisor y receptor.

Proceso

Aquí ambas placas cargan casi el mismo programa y lo único que cambia es la MAC de la otra placa. Cada una guarda en `macOtraPlaca` la MAC de su compañera:

```
// Placa de Derek (04_Bidireccional_Tu_placa): MAC de la placa de Renata
uint8_t macOtraPlaca[6] = {0xB8, 0x1F, 0x3F, 0xDA, 0x18, 0xA8};

// Placa de Renata (04_Bidireccional_Placa_companero): MAC de la placa de Derek
uint8_t macOtraPlaca[6] = {0xDC, 0xDA, 0x0C, 0x26, 0xC3, 0x74};
```

Cada placa registra a la otra como peer, envía un mensaje que incluye su propia MAC (`"Hola, soy ..."`) y, a la vez, recibe y muestra los mensajes de la otra. En el monitor serie se distinguen con prefijos: `>>` para lo que envía y `<<` para lo que recibe.

Resultado

En la placa de Derek:

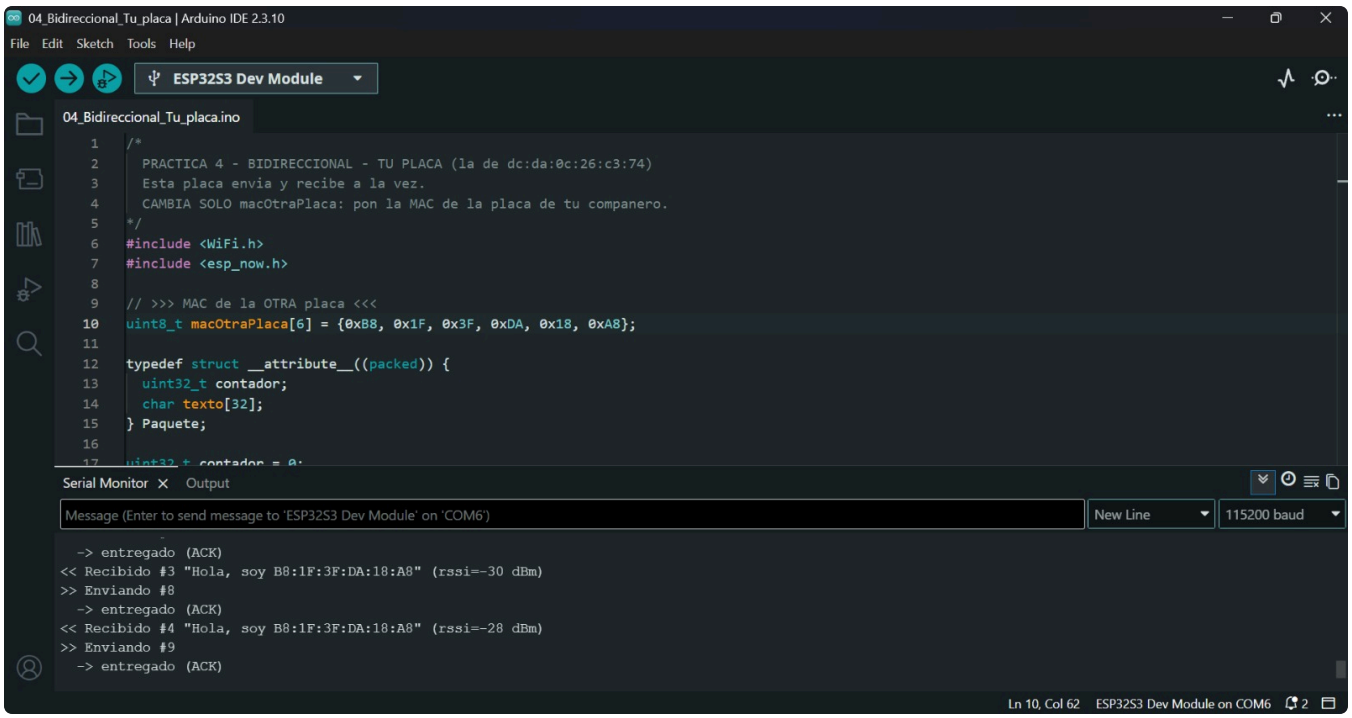
```
<< Recibido #4 "Hola, soy B8:1F:3F:DA:18:A8" (rssi=-28 dBm)
>> Enviando #9
  -> entregado (ACK)
```

Y en la placa de Renata:

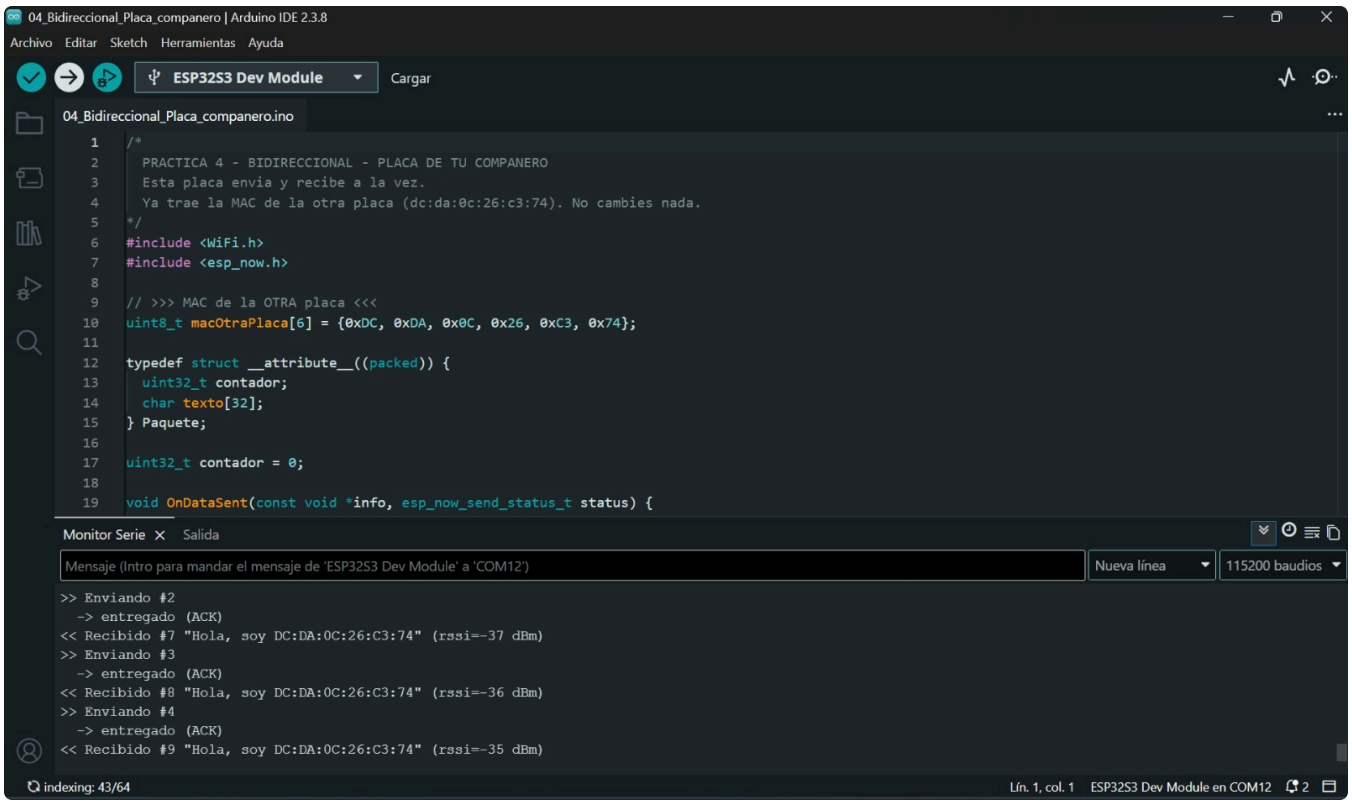
```
<< Recibido #9 "Hola, soy DC:DA:0C:26:C3:74" (rssi=-35 dBm)
>> Enviando #4
  -> entregado (ACK)
```

Cada placa recibió los mensajes de la otra con la MAC correcta y los envíos propios fueron confirmados con ACK, así que la comunicación bidireccional quedó funcionando.

Placa de Derek (COM6):



Placa de Renata (COM12):



Práctica 05: Encriptación

Objetivo

Proteger los mensajes con el cifrado que incluye ESP-NOW, de forma que solo las placas que comparten la clave puedan leerlos.

Proceso

La estructura es la misma que en la práctica 4, pero ahora, al registrar a la otra placa como peer, se activa el cifrado (`peer.encrypt = true`) y se le asigna una clave de 16 caracteres (LMK). Las dos placas deben usar exactamente la misma clave.

```
peer.channel = 0;
peer.ifidx = WIFI_IF_STA;
peer.encrypt = true;
memcpy(peer.lmk, LMK_STR, 16);
if (esp_now_add_peer(&peer) != ESP_OK) Serial.println("No se pudo agregar el
peer cifrado");

Serial.print("Mi MAC: ");
Serial.println(WiFi.macAddress());
```

En el `loop()` cada placa manda un mensaje cifrado cada 3 segundos. Para distinguir las dos placas, usamos textos distintos: una pregunta y su respuesta.

```
void loop() {
  Paquete p;
  p.contador = contador++;
  snprintf(p.texto, sizeof(p.texto), "Como estas?",
WiFi.macAddress().c_str());
  Serial.printf(">> Enviando cifrado #%lu\n", (unsigned long)p.contador);
  esp_now_send(macOtraPlaca, (uint8_t *)&p, sizeof(p));
  delay(3000);
}
```

La placa de Derek manda "Como estas?" y la de Renata responde "Bien y tu?". Los mensajes recibidos se marcan con `[CIFRADO]` en el monitor serie.

Resultado

En la placa de Renata:

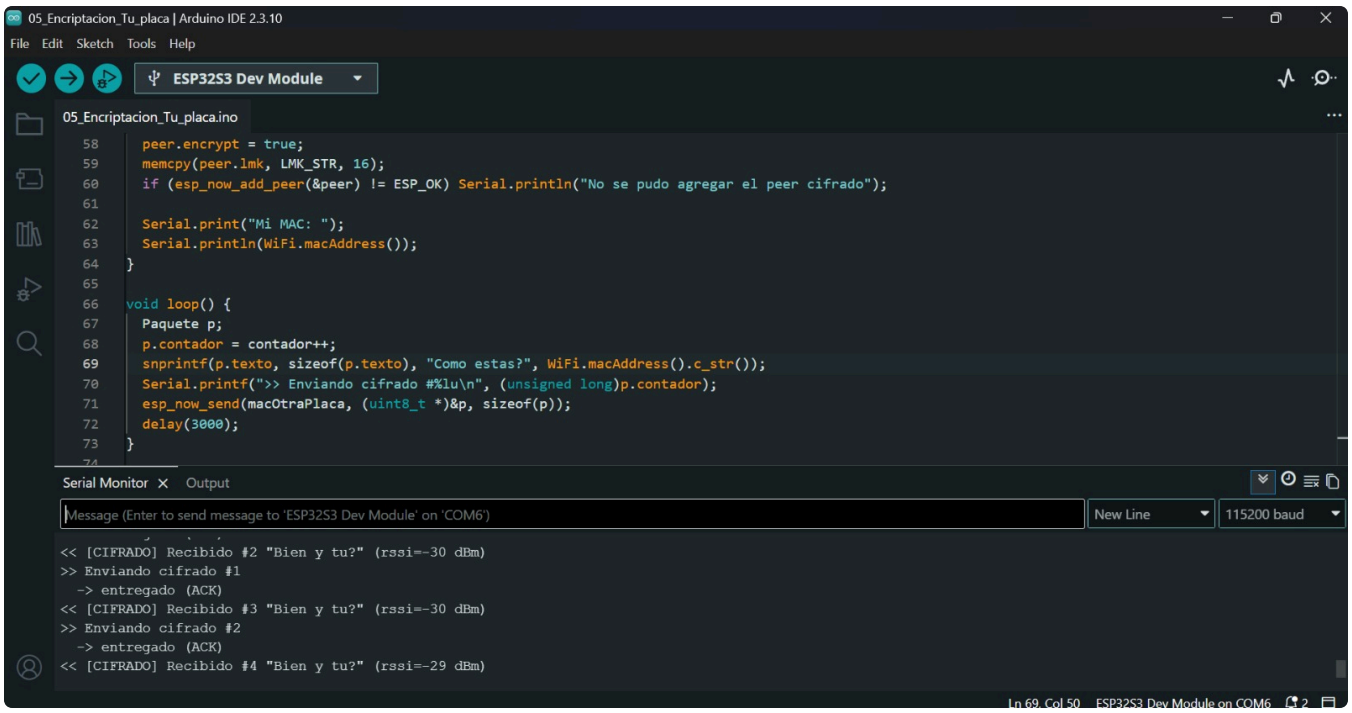
```
<< [CIFRADO] Recibido #2 "Como estas?" (rssi=-34 dBm)
>> Enviando cifrado #4
  -> entregado (ACK)
```

Y en la placa de Derek:

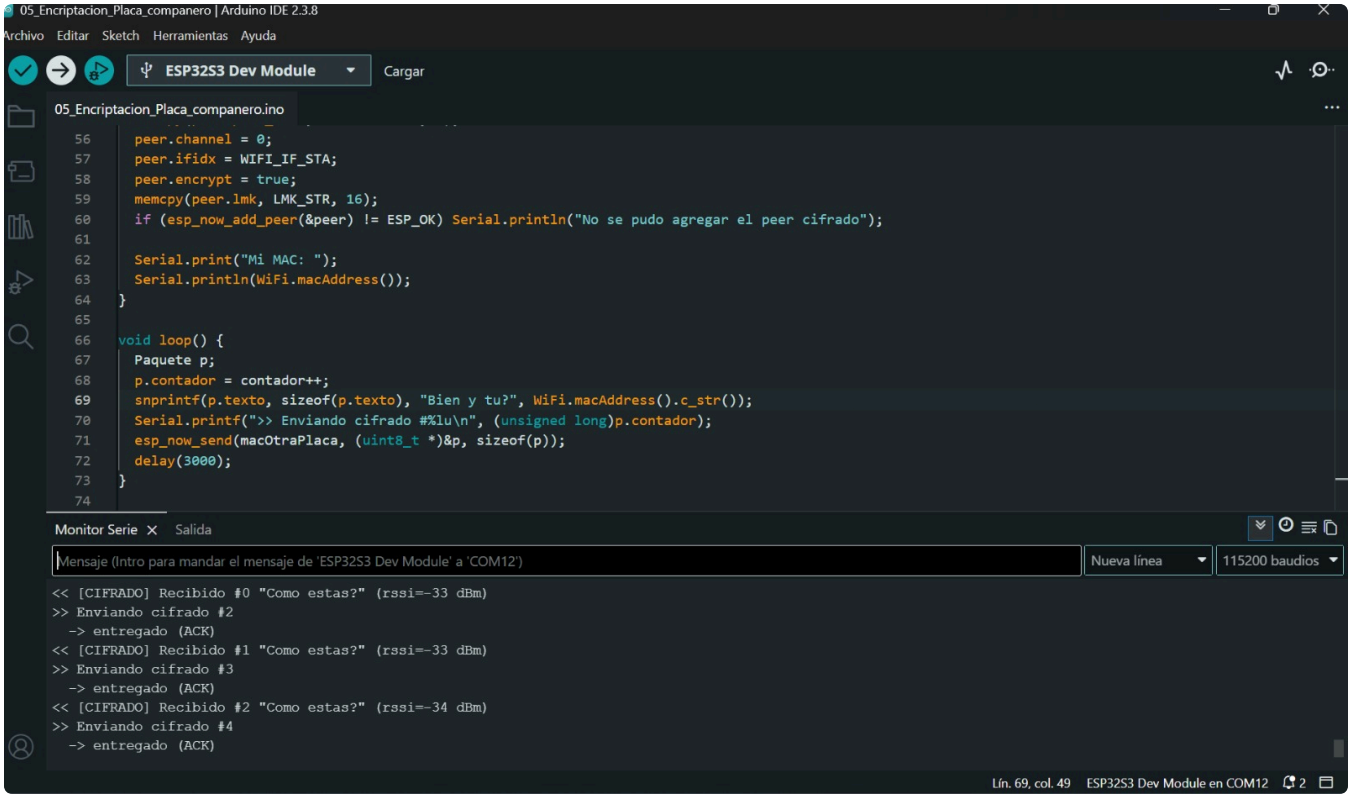
```
<< [CIFRADO] Recibido #4 "Bien y tu?" (rssi=-29 dBm)
```

La conversación cifrada funcionó en ambos sentidos, con ACK en cada envío, y el RSSI se mantuvo alrededor de -30 dBm.

Placa de Derek (COM6):



Placa de Renata (COM12):



Práctica 06: Subir el proyecto a GitHub

Objetivo

Guardar y entregar el código de todas las prácticas en un repositorio de GitHub.

Proceso

Subimos al repositorio todos los sketches mediante la opción "Add files via upload". Para las prácticas con dos placas, se subió un archivo por cada placa (en las prácticas 4 y 5 quedaron como `Placa_1` y `Placa_2`).

Archivos del repositorio:

Práctica	Archivos
00	`00_ObtenerMAC.ino`
01	`01_Distancia_Emisor.ino`, `01_Distancia_Receptor.ino`
02	`02_Unidireccional_Emisor.ino`, `02_Unidireccional_Receptor.ino`
03	`03_Broadcast_Emisor.ino`, `03_Broadcast_Receptor.ino`
04	`04_Bidireccional_Placa_1.ino`, `04_Bidireccional_Placa_2.ino`
05	`05_Encriptacion_Placa_1.ino`, `05_Encriptacion_Placa_2.ino`

Resultado

Los 11 archivos quedaron cargados en el repositorio, con el mismo nombre y orden de las prácticas.

Archivos subidos al repositorio:



Conclusiones

- ESP-NOW permite comunicar placas ESP32 de forma directa, sin router, usando solo la MAC de cada placa.
- El alcance es limitado: con RSSI cercano a -90 dBm la pérdida de paquetes ya era muy alta.
- La comunicación directa a una MAC confirma la entrega con ACK, mientras que el broadcast no lo hace, pero tiene la ventaja de no necesitar ninguna MAC.
- El modo bidireccional se logra registrando a la otra placa como peer en ambas, y el cifrado se activa con una clave compartida de 16 caracteres.
- Trabajar en equipo con dos placas nos permitió probar cada práctica en una situación real de envío y recepción.